

Project Documentation: Mealy Rising Edge Detector Unit

Digital Design Lab

Project Overview

This lab implements a **Mealy Finite State Machine (FSM)** for rising edge detection in Verilog HDL[cite: 31]. Unlike a Moore FSM, the output `tick` in this Mealy design is combinationaly determined by both the current state (`ZERO`) and the active input signal (`level == 1'b1`)[cite: 31]. This allows the output to react immediately to an edge transition without waiting for the next clock boundary[cite: 31].

Specifications & Functionality

- **States:**
 - `ZERO (1'b0)`: Low input tracking state[cite: 31]. Output `tick = 1` combinationaly if `level = 1`[cite: 31]. Transitions to `ONE` on `level = 1`, or remains in `ZERO` if `level = 0`[cite: 31].
 - `ONE (1'b1)`: High input tracking state[cite: 31]. Output `tick = 0`[cite: 31]. Remains in `ONE` while `level = 1`; returns to `ZERO` when `level = 0`[cite: 31].
- **Asynchronous Reset:** Active-high reset immediately forces state register to `ZERO` on `posedge reset`[cite: 31].
- **Mealy Output Logic:** Evaluates directly as `tick = (current_state == ZERO) ^ (level == 1'b1)`[cite: 31].

Port Summary

Port Name	Direction	Type	Width	Description
<code>clk</code>	Input	<code>wire</code>	1	Clock input signal[cite: 31, 32]
<code>reset</code>	Input	<code>wire</code>	1	Asynchronous active-high reset[cite: 31, 32]
<code>level</code>	Input	<code>wire</code>	1	Input level signal to monitor[cite: 31, 32]
<code>tick</code>	Output	<code>reg</code>	1	Combinational pulse output[cite: 31, 32]

Table 1: Mealy Edge Detector Port Interfaces

Verilog Source Code

Design Module (`rising_edge_mealy.v`)

```
module rising_edge_mealy (  
    input wire clk,  
    input wire reset,
```

```

    input  wire  level,
    output reg   tick
);

parameter ZERO = 1'b0;
parameter ONE  = 1'b1;

reg current_state, next_state;

// State Register with Asynchronous Reset
always @(posedge clk or posedge reset) begin
    if (reset)
        current_state <= ZERO;
    else
        current_state <= next_state;
end

% Next State Logic
always @(*) begin
    case (current_state)
        ZERO: begin
            if (level)
                next_state = ONE;
            else
                next_state = ZERO;
        end

        ONE: begin
            if (level)
                next_state = ONE;
            else
                next_state = ZERO;
        end

        default:
            next_state = ZERO;
    endcase
end

// Mealy Output Logic (State + Input dependent)
always @(*) begin
    if ((current_state == ZERO) && (level == 1'b1))
        tick = 1'b1;
    else
        tick = 1'b0;
end

endmodule

```

Testbench Module (tb_rising_edge_mealy.v)

```

`timescale 1ns / 1ps

module tb_rising_edge_mealy;

    // Testbench signals

```

```
reg clk;
reg reset;
reg level;

wire tick;

// Instantiate the DUT
rising_edge_mealy DUT (
    .clk    (clk),
    .reset  (reset),
    .level  (level),
    .tick   (tick)
);

// Clock generation (10ns Period)
initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end

// Test stimulus
initial begin
    // Initial values
    reset = 1'b1;
    level = 1'b0;
    #10;

    reset = 1'b0;
    #10;

    level = 1'b1;
    #20;

    level = 1'b1;
    #20;

    level = 1'b0;
    #20;

    level = 1'b1;
    #20;

    level = 1'b0;
    #20;

    $finish;
end

// Output Display Monitor
initial begin
    $monitor(
        "Time = %0t | clk = %b | reset = %b | level = %b | tick = %b",
        $time,
        clk,
        reset,
        level,
        tick
    );
end
```

```
    );  
    end  
  
endmodule
```

Simulation & Verification

Execution Command

To execute batch simulation using Cadence Xcelium or ModelSim:

```
xrun rising_edge_mealy.v tb_rising_edge_mealy.v
```

Expected Simulation Output

The simulation log confirms that `tick` asserts immediately upon `level` transitioning high while in state `ZERO`[cite: 31, 32]:

```
Time = 0 | clk = 0 | reset = 1 | level = 0 | tick = 0  
Time = 5 | clk = 1 | reset = 1 | level = 0 | tick = 0  
Time = 10 | clk = 0 | reset = 0 | level = 0 | tick = 0  
Time = 20 | clk = 0 | reset = 0 | level = 1 | tick = 1  
Time = 25 | clk = 1 | reset = 0 | level = 1 | tick = 0  
Time = 60 | clk = 0 | reset = 0 | level = 0 | tick = 0  
Time = 80 | clk = 0 | reset = 0 | level = 1 | tick = 1  
Time = 85 | clk = 1 | reset = 0 | level = 1 | tick = 0
```